

CSE 402

University of Asia Pacific

Course Name : Operating System Lab

Lab Report 2

Topic : Shortest Job First Algorithm

Presented by,

Fahad Hasan Mahi
Reg : 23101186
Section : D1

Presented to,

Atia Rahman Orthi
Lecturer, UAP

Problem Statement :

Process ID	Arrival Time (A.T)	Burst Time (B.T)
P1	3	3
P2	2	1
P3	5	2
P4	0	3
P5	1	2

Implement the **Shortest Remaining Time First (SRTF)** CPU scheduling algorithm, the preemptive version of SJF. For the given set of 5 processes, simulate SRTF to calculate Completion Time (CT), Turnaround Time (TAT = CT − AT), Waiting Time (WT = TAT − BT), and Response Time (RT = first execution time − AT) for each process. Also compute the average Turnaround Time and average Waiting Time, and display the results with the Gantt chart.

Source Code :

(For non-preemptive)-

```
def sjf(parameters):
 n = len(parameters)
 completion_time = [0] * n
 turnaround_time = [0] * n
 waiting_time = [0] * n

 remaining = list(parameters)
 gantt = []
 current_time = 0

 while remaining:
 available = [p for p in remaining if p[1] <= current_time]

 # CPU idle: jump to next arrival
 if not available:
 next_arrival = min(p[1] for p in remaining)
 gantt.append(("IDLE", current_time, next_arrival))
 current_time = next_arrival
 continue
```

```
 chosen = min(available, key=lambda p: (p[2], p[1], p[0]))

 start = current_time
 current_time = current_time + chosen[2]
 gantt.append((chosen[0], start, current_time))

 i = parameters.index(chosen)
 completion_time[i] = current_time
 turnaround_time[i] = completion_time[i] - chosen[1]
 waiting_time[i] = turnaround_time[i] - chosen[2]

 remaining.remove(chosen)

 return {
 'process_id': parameters,
 'completion_time': completion_time,
 'turnaround_time': turnaround_time,
 'waiting_time': waiting_time,
 'gantt': gantt
 }


parameters = [
 ('P1', 3, 3), ('P2', 2, 1), ('P3', 5, 2),
 ('P4', 0, 3), ('P5', 1, 2)
]

result = sjf(parameters)

print("Process\tArrival\tBurst\tCompletion\tTurnaround\tWaiting")
for i, (pid, arrival, burst) in enumerate(result['process_id']):
 print(f"{pid}\t{arrival}\t{burst}\t{result['completion_time'][i]}\t\t"
 f"{result['turnaround_time'][i]}\t\t{result['waiting_time'][i]}")

print("\nGantt Chart:")
for name, start, end in result['gantt']:
 print(f"[{name}: {start}->{end}]", end=" ")
print()

avg_turnaround_time = sum(result['turnaround_time']) / len(result['turnaround_time'])
avg_waiting_time = sum(result['waiting_time']) / len(result['waiting_time'])

print(f"\nAverage Turnaround Time: {avg_turnaround_time}")
print(f"Average Waiting Time: {avg_waiting_time}")
```

(For SRTF/preemptive)-

```
def srtf(processes):
 n = len(processes)
 pid = [p[0] for p in processes]
 arrival = [p[1] for p in processes]
 burst = [p[2] for p in processes]
 remaining = burst.copy()

 completion = [0] * n
 turnaround = [0] * n
 waiting = [0] * n
 response = [0] * n
 start_time = [-1] * n
 is_completed = [False] * n

 current_time = 0
 completed = 0
 gantt_chart = []
 previous_pid = None

 while completed < n:
 shortest_idx = -1
 min_remaining = float('inf')

 for i in range(n):
 if (arrival[i] <= current_time
 and not is_completed[i]
 and remaining[i] < min_remaining):
 min_remaining = remaining[i]
 shortest_idx = i

 # CPU idle
 if shortest_idx == -1:
 if previous_pid != "IDLE":
 gantt_chart.append(("IDLE", current_time, current_time + 1))
 previous_pid = "IDLE"
 current_time += 1
 continue

 if start_time[shortest_idx] == -1:
 start_time[shortest_idx] = current_time

 current_time += 1 # run 1 unit, then re-check
 remaining[shortest_idx] -= 1

 if previous_pid != pid[shortest_idx]:
 gantt_chart.append((pid[shortest_idx], current_time - 1, current_time))
```

```
 else:
 gantt_chart[-1] = (pid[shortest_idx], gantt_chart[-1][1], current_time)
 previous_pid = pid[shortest_idx]

 if remaining[shortest_idx] == 0:
 is_completed[shortest_idx] = True
 completion[shortest_idx] = current_time
 turnaround[shortest_idx] = completion[shortest_idx] - arrival[shortest_idx]
 waiting[shortest_idx] = turnaround[shortest_idx] - burst[shortest_idx]
 completed += 1

 for i in range(n):
 response[i] = start_time[i] - arrival[i]

 return {
 'pid': pid, 'arrival': arrival, 'burst': burst,
 'completion': completion, 'turnaround': turnaround,
 'waiting': waiting, 'response': response,
 'gantt_chart': gantt_chart
 }


processes = [
 ('P1', 3, 3), ('P2', 2, 1), ('P3', 5, 2),
 ('P4', 0, 3), ('P5', 1, 2)
]

result = srtf(processes)

print("Process\tArrival\tBurst\tCompletion\tTurnaround\tWaiting\tResponse")
for i in range(len(result['pid'])):
 print(f"{result['pid'][i]}\t{result['arrival'][i]}\t{result['burst'][i]}\t"
 f"{result['completion'][i]}\t\t{result['turnaround'][i]}\t\t"
 f"{result['waiting'][i]}\t\t{result['response'][i]}")

print("\nGantt Chart:")
for name, start, end in result['gantt_chart']:
 print(f"[{name}: {start}->{end}]", end=" ")
print()

avg_turnaround = sum(result['turnaround']) / len(result['turnaround'])
avg_waiting = sum(result['waiting']) / len(result['waiting'])
avg_response = sum(result['response']) / len(result['response'])

print(f"\nAverage Turnaround Time: {avg_turnaround}")
print(f"Average Waiting Time: {avg_waiting}")
print(f"Average Response Time: {avg_response}")
```

Output Screenshot :

(for non-preemptive)

```
sh-5.3$ /usr/bin/python "/home/fahad369/Documents/OS_Lab/CSE_402_Assignment_materials/Lab 2/SJF_(non_preempt
ive).py"
Process Arrival Burst   Completion      Turnaround      Waiting
P1      3       3       11              8               5
P2      2       1       4               2               1
P3      5       2       8               3               1
P4      0       3       3               3               0
P5      1       2       6               5               3

Gantt Chart:
[P4: 0->3] [P2: 3->4] [P5: 4->6] [P3: 6->8] [P1: 8->11]

Average Turnaround Time: 4.2
Average Waiting Time: 2.0
sh-5.3$
```

(for SRTF/preemptive)

```
sh-5.3$ /usr/bin/python "/home/fahad369/Documents/OS_Lab/CSE_402_Assignment_materials/Lab 2/SJF_(preemptive)
.py"
Process Arrival Burst   Completion      Turnaround      Waiting Response
P1      3       3       11              8               5               5
P2      2       1       3               1               0               0
P3      5       2       8               3               1               1
P4      0       3       4               4               1               0
P5      1       2       6               5               3               3

Gantt Chart:
[P4: 0->2] [P2: 2->3] [P4: 3->4] [P5: 4->6] [P3: 6->8] [P1: 8->11]

Average Turnaround Time: 4.2
Average Waiting Time: 2.0
Average Response Time: 1.8
sh-5.3$
```

Comparison between FCFS, SJS (non preemptive), SJF (preemptive) :

Numeric (FCFS values taken from Report-1) :

Process	FCFS (TWT/WT)	SJF (TWT/WT)	SRTF (TWT/WT)
P1	6/3	8/5	8/5
P2	4/3	2/1	1/0
P3	6/4	3/1	3/1
P4	3/0	3/0	4/1
P5	4.2	5/3	5.3
Average	4.6 /2.4	4.2/2.0	4.2/2.0
Average Response	2.4	2.0	1.8

Feature Comparison :

Feature	FCFS	SJS (non-preemptive)	SJF (preemptive)
Selection	Arrival Order	Shortest Burst Time	Shortest Remaining Time
Decision Point	CPU free	CPU free	Every arrival/completion
Starvation	No	Yes	Yes
Convey Effect	Possible	Less Likely	No
Context Switching	Low	Low	High
Complexity	Simple	Moderate	More Complex

Discussion :

The two separate programs show the core difference between the variants: non-preemptive SJF lets P4 run uninterrupted (0-3), while SRTF preempts P4 at t=2 for the shorter P2 and resumes P4 at t=3. Both beat FCFS (avg WT 2.0 vs 2.4), and SRTF additionally achieves the best response time (1.8) at the cost of extra context switches. Both SJF variants still carry starvation risk for long processes.

Conclusion :

Through this lab, it's verified that SJF and its preemptive variant SRTF outperform FCFS in waiting and turnaround time for the given process set, with SRTF additionally minimizing response time by allowing preemption. However, this improvement requires more context switching and still carries the risk of starvation for long processes. Therefore, the choice of scheduling algorithm ultimately depends on the system's requirements, workload, and desired performance.

Github repository : https://github.com/fhmahi957/CSE_402_Assignment_materials.git